

Submission Worksheet

Submission Data

Course: IT114-004-S2026

Assignment: IT114 Milestone 3 - Generic Project Submission

Student: Stefan I. (si329)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 5/7/2026 7:15:53 PM

Updated: 5/7/2026 11:06:14 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-3-generic-project-submission/grading/si329>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-3-generic-project-submission/view/si329>

Instructions

1. Refer to the Milestone 3 requirements for your approved project proposal and complete the required features before filling this out.
 - Project proposal docs:
 - [Rock Paper Scissors](#)
 - [Hangman](#)
 - [Trivia](#)
 - [Basic Battleship](#)
 - [Drawing](#)
 - [Go Fish](#)
 - This worksheet includes project hints, but your full project proposal document is the source of truth for project-specific requirements.
2. Ensure all code snippets include your UCID, date, and a brief description of what the code does.
3. Switch to the Milestone3 branch.
 1. `git checkout Milestone3`
 2. `git pull origin Milestone3`
4. Fill out the worksheet below as you test and demo with 3+ clients in the same session.
5. Once finished, click "Submit and Export".
6. Add the generated PDF somewhere inside your repository and move it to GitHub.
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On GitHub merge the pull request from Milestone3 to main
7. Upload the same PDF to Canvas.
8. Sync local.
 1. `git checkout main`
 2. `git pull origin main`

◦ Review the details and validation criteria for each task.

Section #1: (1 pt.) Core Ui

Progress: 100%

☰ Task #1 (0.50 pts.) - Connection/Details Panels

Progress: 100%

Details:

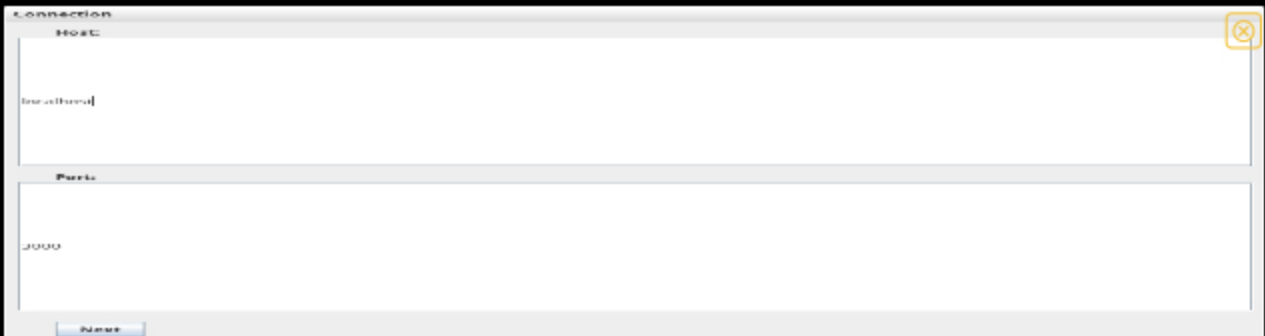
- Show a valid connection flow from UI to connected session
 - Connection fields: host and port
 - Authentication field: username

🖼 Part 1:

Progress: 100%

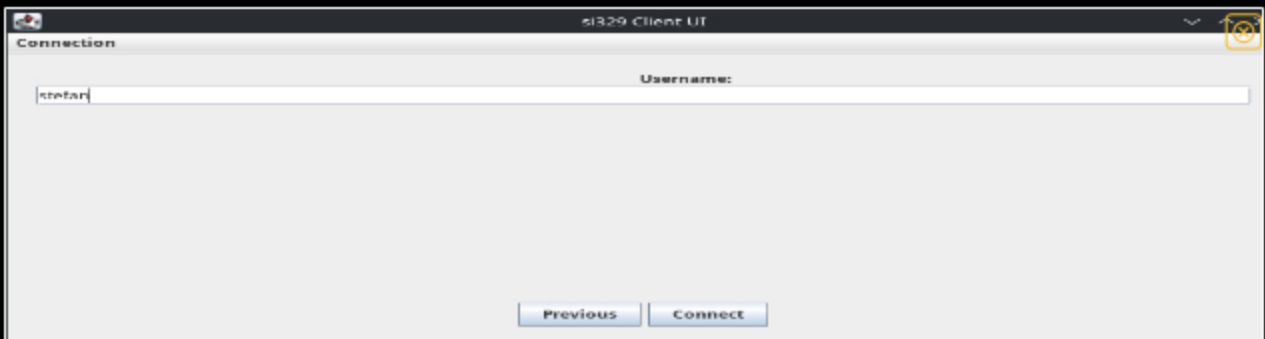
Details:

- Show the connection panel with valid data
- Show the user details panel with valid data
- Include code snippets showing the flow from UI capture to connection handling



The screenshot shows a UI window titled "Connection". It contains two text input fields. The first field is labeled "Host:" and contains the text "localhost". The second field is labeled "Port:" and contains the text "3000". Below these fields is a button labeled "Connect".

Shows the connection panel where a localhost and port can be entered



The screenshot shows a UI window titled "51329 Client UI" with a sub-header "Connection". It contains a text input field labeled "Username:" with the text "stefan" entered. Below the field are two buttons: "Previous" and "Connect".

Shows the panel where you can authenticate

```
// start region: IClientCommands implementation
// Methods in this block are the public command surface used by the UI.

@Override
public boolean connectToServer(String host, int port, String name) {
    try {
        ValidationUtils.requireNotBlank(name, errorMessage: "Name cannot be blank");
        myUser.setClientName(name.trim());
        if (!connect(host, port)) {
            // ...
        }
    }
}
```

```

        }
        sendConnectionData(myuser.getClientName());
        return true;
    } catch (ValidationException e) {
        LoggerUtil.INSTANCE.warning(TextFX.colorize(e.getMessage(), color.YELLOW));
        emitUISystemMessage(e.getMessage());
    } catch (IOException e) {
        LoggerUtil.INSTANCE.severe("Failed to send connection payload: " + e.getMessage());
        emitUISystemMessage(message: "Failed to complete connection handshake.");
    }
    return false;
}

```

connectoServer method

```

connectionView = new ConnectionView((host, port) -> {
    pendingHost = host;
    pendingPort = port;
    userDetailsView.clearError();
    showScreen(Screen.USER);
});

```

shows connectionview capture in clientui

```

userDetailsView = new UserDetailsView(
    () -> showScreen(Screen.CONNECTION),
    name -> {
        userDetailsView.clearError();
        boolean connecting = Client.INSTANCE.connectToServer(pendingHost, pendingPort, name);
        if (connecting) {
            showScreen(Screen.CHAT);
        } else {
            userDetailsView.setError(message: "Connection attempt failed. Check host/port and try again.");
        }
    }
);

chatGameView = new ChatGameView();

```

captures username

```

private void sendConnectionData(String clientName) throws IOException {
    ConnectionPayload payload = new ConnectionPayload();
    payload.setPayloadType(PayloadType.CLIENT_CONNECT);
    payload.setClientName(clientName);
    sendToServer(payload);
}

```

Builds and sends payload type

```

// start region: IClientCommands implementation
// Methods in this block are the public command surface used by the UI.
@Override
public boolean connectToServer(String host, int port, String name) {
    try {
        ValidationUtils.requireNotBlank(name, errorMessage: "Name cannot be blank");
        myUser.setClientName(name.trim());
        if (!connect(host, port)) {
            return false;
        }
        sendConnectionData(myuser.getClientName());
        return true;
    } catch (ValidationException e) {
        LoggerUtil.INSTANCE.warning(TextFX.colorize(e.getMessage(), color.YELLOW));
        emitUISystemMessage(e.getMessage());
    } catch (IOException e) {
        LoggerUtil.INSTANCE.severe("Failed to send connection payload: " + e.getMessage());
        emitUISystemMessage(message: "Failed to complete connection handshake.");
    }
    return false;
}

```

This method is invoked under the processPayload method, as CASE: CLIENT_CONNECT



Saved: 5/7/2026 7:42:47 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

The client enters the information, eg the IP address, port number and username into the fields, and upon pressing the button, whatever is entered in the fields is collected and stored as pendingHost/pendingPort as well as name. This is then fed into Client.INSTANCE.connectoServer(which takes in these values here). A payload is then built and this is then piped up into the server. ServerThread receives this, pipes it into the cases in ProcessPayload and uses this information to establish a connection. Client name is then set using setClientName(), and onInitalized is triggered which registers the client and adds it to the "list", gives an ID (to prevent the issue of multiple people sharing the same name), and it then calls a broadcast to all the other clients.



Saved: 5/7/2026 7:42:47 PM

≡ Task #2 (0.50 pts.) - Ready Panel

Progress: 100%

Details:

- Show a ready-check UI that supports session lifecycle
 - Should have a button to mark ready
 - Should show how many people are ready out of the number required to start
 - Should reappear when a session ends
- Session creator rules
 - The first person to invoke the ready-check is the session creator
 - Only the session creator can adjust session settings
- Project hints for settings controlled here:
 - RPS: extra choices and cooldown toggle
 - Hangman: restore-strikes toggle and hard mode toggle
 - Trivia: category selection
 - Go Fish: extra decks and wildcard/joker behavior
 - Battleship / Drawing: any project-specific session options exposed in your proposal

🖼 Part 1:

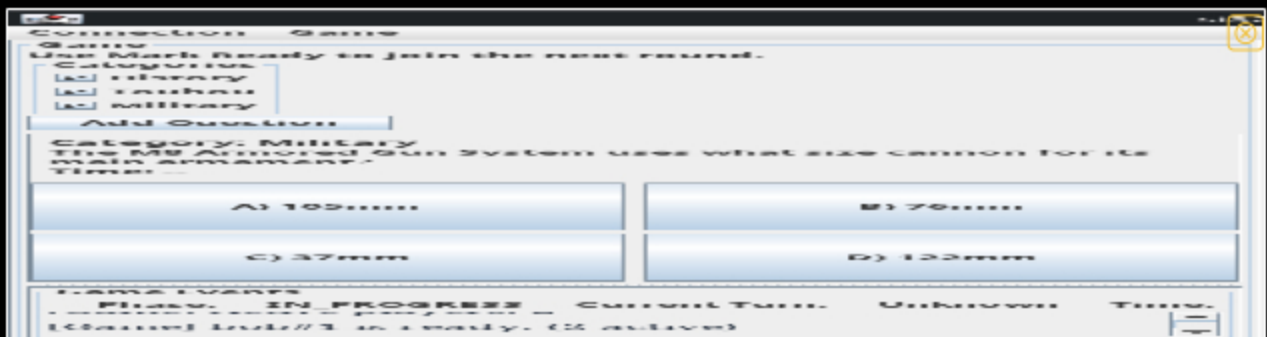
Progress: 100%

Details:

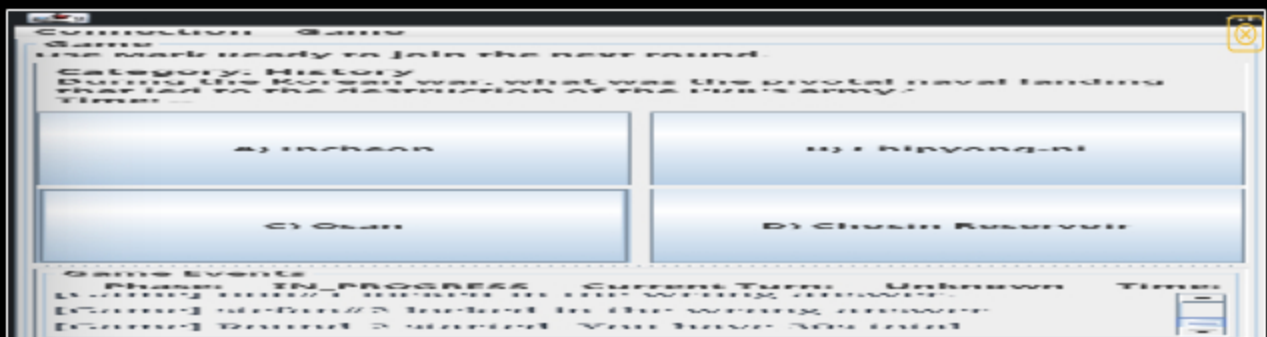
- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)
- Show host-only settings controls and evidence that non-host users cannot modify them



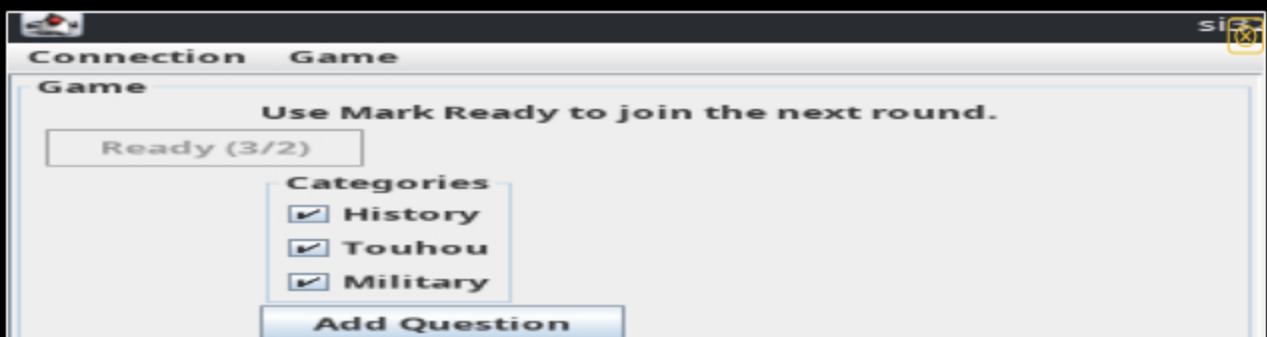
Ready button with only one person ready



Host panel shown only to the host



Non host does not have access to host options



Three clients are ready, option is greyed out because this client is already connected.

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI
- Briefly explain how host-only settings are enforced server-side

Your Response:

When the player clicks "Ready", `Client.INSTANCE.sendReadySignal()` is called. All this does is reference another method, `sendReady()`, which builds a new payload of type `READY`. This is then sent along the server in `sendToServer(payload)`, which again similarly receives this payload, processes it in `CASE READY`, calling `processReady()`, and then invoking the method `handleReady()`, which exists in its entirety in `GameServer` but essentially what it does is adds them to the "list" that compiles the ready players. The server then broadcasts out a payload of type `PLAYER_READY_STATUS`, as well as `GAME_PHASE_SYNC`. This then tells the clients that they need to update their UIs, after which they invoke the `processReadyStatus()` method, which fires `emitUiPlayerStatusUpdated()` and changes their badge. Host only settings are enforced when the first player invokes `handleReady()`, which adds this player's ID to a variable called `creatorClientId`. When any client attempts to toggle a category, the server simply invokes `ValidationUtils` before processing the toggle. If the sender is not the creator, this throws a `ValidationException`.

Section #2: (2 pts.) Project Ui

Progress: 100%

≡ Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

- Show the username and id of each user
- Show the current points of each user
- Users should follow this ordering rule across each client
 - Turn-based games: turn order is required
 - Battleship
 - Go Fish
 - Hangman
 - Non-turn-based games: points order is required
 - Drawing
 - Rock Paper Scissors
 - Trivia

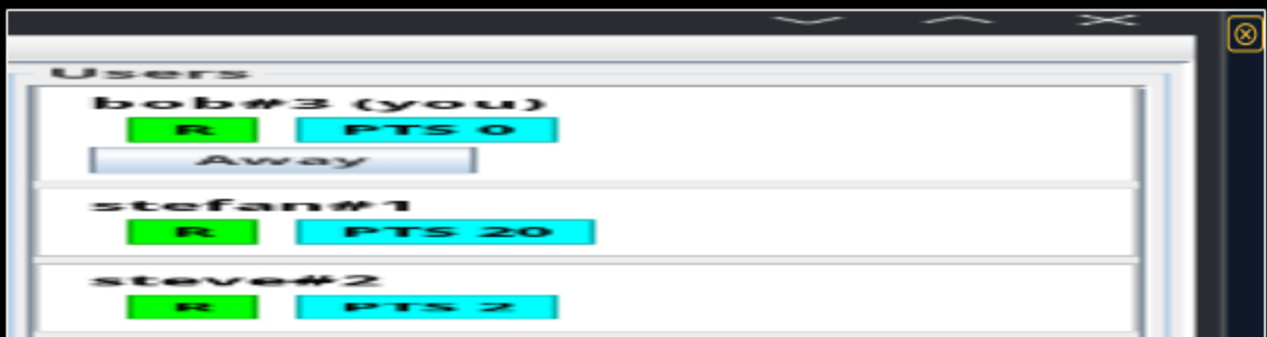
- If points are tied, use a deterministic tie-break (for example username)
- Show project-specific status indicators as applicable
 - Turn indicator or drawer indicator
 - Locked-in/pending indicator
 - Eliminated indicator (if your project supports elimination)
 - Project-specific stats (examples: ships remaining, hit/miss %, hand count)

Part 1:

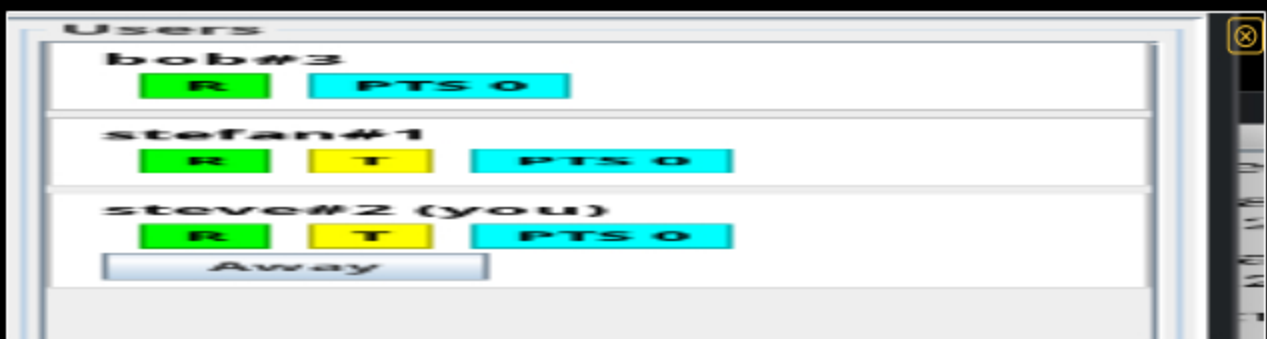
Progress: 100%

Details:

- Show various examples of points (3+ clients visible)
 - Include code snippets showing the code flow for this from server-side to UI
- Show that sorting/order is maintained across clients
 - Include code snippets showing the code that handles this
 - Include one runtime example and one code snippet that explicitly proves your required ordering rule (turn-based = turn order, non-turn-based = points order)
- Show project-specific indicators/stats required by your proposal
 - Include code snippets showing the code flow for this from server-side to UI



Shows different points per user (small mistake I have to fix in the code, it needs to iterate DOWN from 10 per each person)



Shows different user states (t Indicates an answer was submitted)

```
private void broadcastPlayerPoints(ServerThread player) {
    if (player == null) {
        return;
    }
}
```

```

    }
    Server.INSTANCE.sendOrDisconnect(
        serverThread -> serverThread.sendPlayerPoints(player.getClientId(), player.getPoints());
    }
}

```

broadcastPlayerPoints

```

protected boolean sendPlayerPoints(long clientId, int points) {
    PointsPayload payload = new PointsPayload();
    payload.setPayloadType(PayloadType.POINTS);
    payload.setClientId(clientId);
    payload.setPoints(points);
    return sendToClient(payload);
}

```

sendplayerpoints

```

case POINTS:
    processPoints(payload);
    break;

```

The payload is received in process payload, which then invokes the processPoints method.

```

protected void processPoints(PayloadType payload) {
    // If the client is not connected, return false
    if (!Server.INSTANCE.isConnected(serverThread)) {
        return false;
    }
    // Get the client ID and points from the payload
    long clientId = ((PointsPayload) payload).getClientId();
    int points = ((PointsPayload) payload).getPoints();
    // Check if the client is already in the game
    if (gameState.containsKey(clientId)) {
        // If the client is already in the game, update their points
        gameState.put(clientId, gameState.get(clientId) + points);
        // If the client is not in the game, add them
        if (!gameState.containsKey(clientId)) {
            gameState.put(clientId, points);
        }
    }
    // Broadcast the updated points to all clients
    Server.INSTANCE.sendToAll(gameState, "All users' points are: " + gameState);
    return true;
}

// If the client is not connected, return false
if (!Server.INSTANCE.isConnected(serverThread)) {
    return false;
}
// Get the client ID and points from the payload
long clientId = ((PointsPayload) payload).getClientId();
int points = ((PointsPayload) payload).getPoints();
// Check if the client is already in the game
if (gameState.containsKey(clientId)) {
    // If the client is already in the game, update their points
    gameState.put(clientId, gameState.get(clientId) + points);
    // If the client is not in the game, add them
    if (!gameState.containsKey(clientId)) {
        gameState.put(clientId, points);
    }
}
// Broadcast the updated points to all clients
Server.INSTANCE.sendToAll(gameState, "All users' points are: " + gameState);
return true;
}

```

Shows the full method for process points

```

/** Notifies all connected clients of a player's turn-taken status. */
private void broadcastTurnStatus(long clientId, boolean hasTakenTurn) {
    Server.INSTANCE.sendOrDisconnect(serverThread -> serverThread.sendTurnStatus(clientId, hasTakenTurn));
}

```

Shows broadcast turn status which operates similarly to broadcastPlayerPoints.

```
protected boolean sendTurnStatus(long clientId, boolean hasTakenTurn) {
    BoolPayload payload = new BoolPayload();
    payload.setPayloadType(PayloadType.PLAYER_TURN_STATUS);
    payload.setClientId(clientId);
    payload.setValue(hasTakenTurn);
    return sendToClient(payload);
}
```

Shows send turn status, which actually constructs the "packet" and sends it over

```
private void processTurnStatus(Payload payload) {
    if (!payload instanceof BoolPayload) {
        LoggerUtil.INSTANCE.warning("Expected BoolPayload for PLAYER_TURN_STATUS, got: " + payload.getClass());
        return;
    }
    BoolPayload bp = (BoolPayload) payload;
    if (bp.getClientId() == Constants.DEFAULT_CLIENT_ID) {
        knownUsers.forEach((key, user) -> user.setTurnTaken(turnTaken: false));
        emitUiAllPlayerStatusesReset();
        return;
    }
    User user = knownUsers.get(bp.getClientId());
    if (user == null) {
        return;
    }
    user.setTurnTaken(bp.getValue());
    emitUiPlayerStatusUpdated(user);
    if (isLocalPlayer(user.getClientId()) && bp.getValue()) {
        emitUiPlayerTurnCompleted(user.getClientId());
    }
}
```

Shows how it this is then processed in processTurnStatus



Saved: 5/7/2026 9:32:48 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting/order
- Briefly explain the code flow for server-side to UI of your project-specific indicators/stats

Your Response:

The server broadcasts PLAYER_TURN_STATUS & POINTS payloads using the broadcastPlayerPoints() function. This calls the sendPlayerPoints() function, which creates a new payload of type POINTS, and then sends it back down to the client. This is then received by the client, that processes the payload and then uses ProcessPoints(), to change what it has on its end regarding its own points as well as other player's points, it then calls emitUiPlayerStatusUpdated to set the points flow.

Similar to the points payload, the project-specific indicators are if the turn was taken(? , not sure what else it could be). This follows the same general structure, a broadcast is called, broadcastTurnStatus(), which sends a PLAYER_TURN_STATUS payload, this payload is received by processpayload, which triggers processTurnStatus() in client takes this payload in and then, updates the turntaken value on user, and calls emitUiPlayerStatusUpdated() to reflect that the user has taken their turn.



Saved: 5/7/2026 9:32:48 PM

≡ Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

- Show the event history/messages relevant to your project actions
 - Reuse and extend Milestone 2 event patterns
- Show scoring/points related messages
- Show round/turn/game progress messages (for example turn ownership, drawer, lock-in result, elimination)
- Show project timer(s) by explicit name (for example turn timer or round timer)
 - Do not use vague wording like "timer as applicable"

🖼 Part 1:

Progress: 100%

Details:

- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI
- Show the project timer behavior while session is active

```
[Game] Round 2 started. You have 30s total.  
[Game] stefan#1 locked in their answer.  
[Game] steve#2 locked in their answer.  
[Turn] steve#2 completed their turn.  
[Game] bob#3 locked in their answer.  
[Game] Round ended.  
[Game] The correct answer was: c  
[Game] stefan#1 locked in the correct answer.  
[Game] steve#2 locked in the wrong answer.  
[Game] bob#3 locked in the wrong answer.  
[Points] stefan#1 now has 20 point(s).  
[Game] stefan#1 got 10 points!  
[Game] Round 3 started. You have 30s total.
```

Shows various game events that pertain to the project (wrong & right answers, points, etc, turns being taken)

```
private void broadcastPlayerPoints(ServerThread player) {  
    if (player == null) {  
        return;  
    }  
    Server.INSTANCE.sendOrDisconnect(  
        serverThread -> serverThread.sendPlayerPoints(player.getClientId(), player.getPoints());  
    )  
}  
  
private void unicastPlayerPoints(ServerThread target, long clientId, int points) {  
    if (target == null) {  
        return;  
    }  
    Server.INSTANCE.unicast(target, serverThread -> serverThread.sendPlayerPoints(clientId, points));  
}
```

Broadcast and unicast methods for player points

```
private void processPoints(Payload payload) {  
    if (!payload instanceof PointsPayload) {  
        Logger.INSTANCE.warning("Expected PointsPayload for POINTS confirmation, got: " + payload.getClientId());  
        return;  
    }  
    long clientId = payload.getClientId();  
    int points = ((PointsPayload) payload).getPoints();  
    Server.INSTANCE.unicast(serverThread -> serverThread.sendPlayerPoints(clientId, points));  
}
```

This works similarly to the last function. Player points is responsible for facilitating and recording who has one point total

Game Events

Phase: READY Time: 20s

the next session.

Shows the timer in the game for a ready before the session. The same bracket is used

Shows almost all timehandling methods in GameServer

```
@Override
public void onGameTimerUpdated(TimerType timerType, int secondsRemaining) {
    if (timerType == null) {
        return;
    }
    String display = secondsRemaining < 0 ? "N/A" : String.format("%ds", secondsRemaining);
    timeValue.setText(display);
}
```

Shows the method that handles time in GameEventsView

```
private void processGameTimerSync(Payload payload) {
    if (!(payload instanceof TimerPayload)) {
        LoggerUtil.INSTANCE.warning("Expected TimerPayload for GAME_TIMER_SYNC, got: " + payload.getClass());
        return;
    }
    TimerPayload tp = (TimerPayload) payload;
    TimerType timerType = tp.getTimerType();
    if (timerType == null) {
        LoggerUtil.INSTANCE.warning(message: "Received GAME_TIMER_SYNC with blank timer type.");
        return;
    }
    emitUiGameTimerUpdated(timerType, tp.getSecondsRemaining());
}
```

Shows processGameTimerSync, which takes in the payload and processes it in the client

[illegible]

on round end is responsible for facilitating a good chunk of the points, correct/wrong answer interactions

 Saved: 5/7/2026 9:51:26 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI
- Briefly explain how timer data/state is synchronized

- Details:**

 - Briefly explain the code flow for generating these messages and getting them onto the UI
 - Briefly explain how timer data/state is synchronized

Your Response:

1. `onRoundEnd()` is responsible for calling most of the broadcasts pertaining to points and right and wrong answers. It simply broadcasts them to the server, which then puts them in the chat and then also calls a specific broadcast for `broadcastPlayerPoints` which as has been stated is sent down to the client, processed and the client updates the UI to reflect this. The timer data is broadcasted similarly to each client through a `TimerPayload`, which then takes this data in and processes it using `processGameTimerSync()`, which updates the UI to reflect the information that was received.

 Saved: 5/7/2026 9:51:26 PM

≡ Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

- Show the core interactive game area required by your project
- Project hints:
 - Battleship: battlefield grid, hit/miss and local ship visibility rules
 - Drawing: drawing board, guess entry area, word/blank visibility rules
 - Go Fish: target-player selection and card/suit selection with confirmation flow
 - Hangman: visual hangman state, letter grid, and full-word guess input

- ### Details:
- Show the core interactive game area required by your project
 - Project hints:
 - Battleship: battlefield grid, hit/miss and local ship visibility rules
 - Drawing: drawing board, guess entry area, word/blank visibility rules
 - Go Fish: target-player selection and card/suit selection with confirmation flow
 - Hangman: visual hangman state, letter grid, and full-word guess input

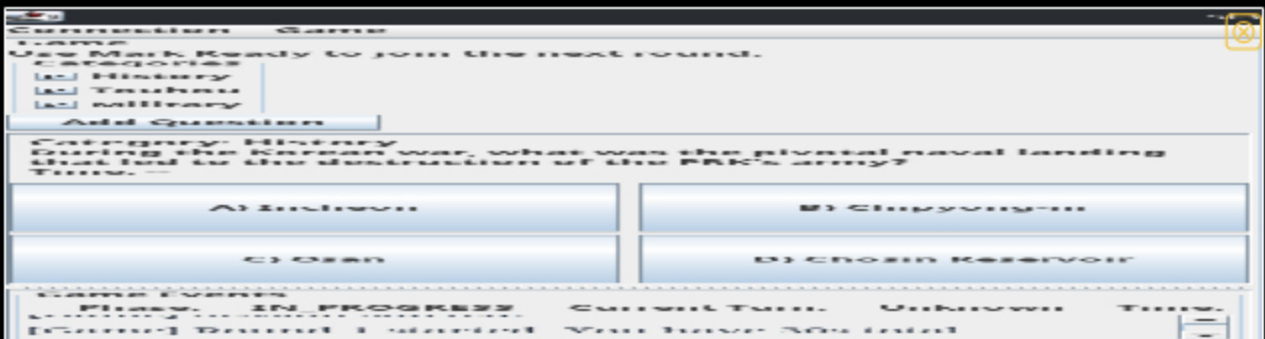
- Trivia: category/question/answers with lock-in behavior
- Ensure UI interaction rules are enforced (for example role/turn/phase restrictions)

Part 1:

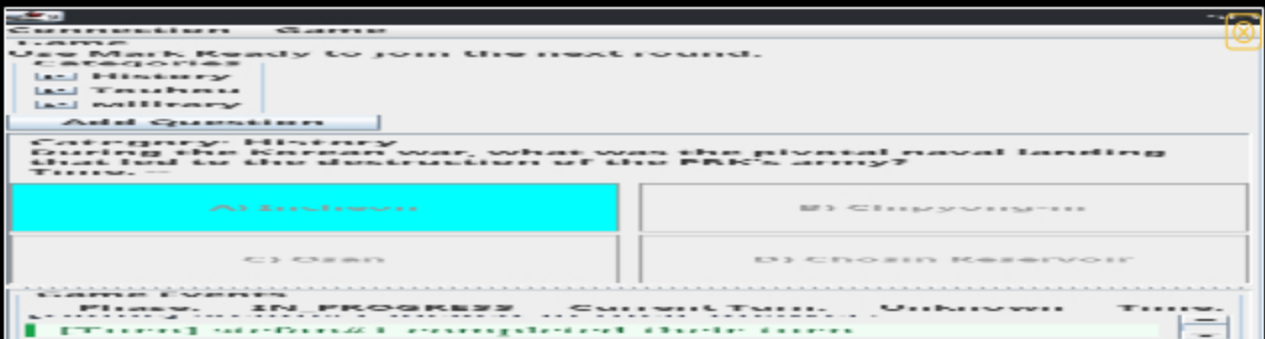
Progress: 100%

Details:

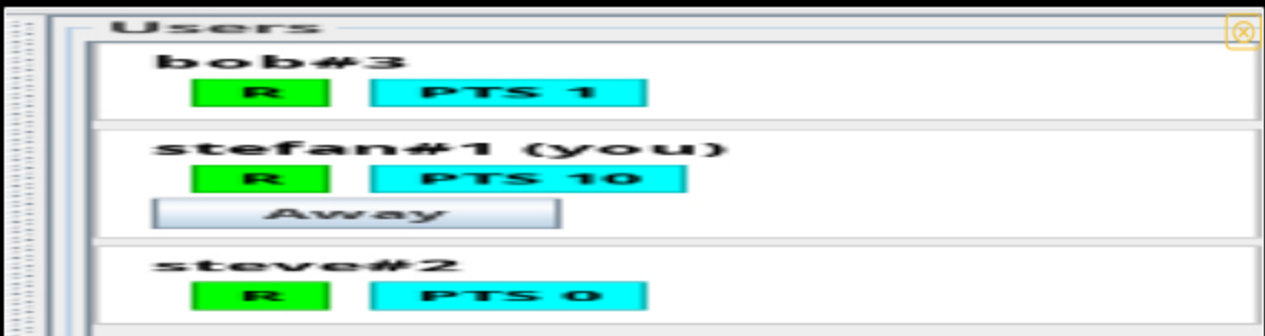
- Show a few examples of interactions in this area across 3+ clients
- Include code snippets showing UI-to-server and server-toUI flow for core interactions
- Include one example where interaction is blocked due to a valid rule (for example not your turn, invalid action, phase mismatch)



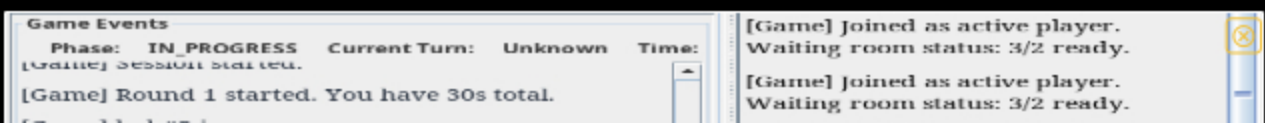
Shows the choices of subjects you have



Selected a choice, now the options are greyed out and you can't select another



Shows the points reflecting the right/wrong answers (interaction across 3 clients)



[Game] bob#3 is away.
[Game] bob#3 is back.
[Game] bob#3 is away.
[Game] bob#3 is back.
[Game] bob#3 is away.
[Game] bob#3 is back.
[Game] bob#3 is away.

[Game] Joined as active player.
Waiting room status: 3/2 ready.
[Game] Joined as active player.
Waiting room status: 3/2 ready.
[Game] Player is away and cannot perform actions

Send

can't select when away

```
JPanel answersPanel = new JPanel(new GridLayout(2,2,6,6));
String[] labels = {"A","B","C","D"};
for (int i = 0; i < 4; i++) {
    final String choice = labels[i];
    answerButtons[i] = new JButton(choice);
    answerButtons[i].setEnabled(false);
    answerButtons[i].addActionListener(event -> {
        try {
            client.sendAnswerSignal(choice);
            lockInAnswer(choice);
        }
        catch (ValidationException e) {
            statusLabel.setText(e.getMessage());
        }
    });
    answersPanel.add(answerButtons[i]);
}
```

Shows how the panel is made

```
@Override
public void sendAnswerSignal(String choice) throws ValidationException {
    try {
        sendAnswer(choice);
    }
    catch (IOException e) {
        LoggerUtil.INSTANCE.warning("Failed to send answer signal:" + e.getMessage());
        emitUISystemMessage(message: "Failed to send answer signal.");
    }
}
```

send answer signal

```
private void sendAnswer(String answer) throws ValidationException {
    String validationException = answer == null ? " : answer error" : "";
    System.out.println("[DEBUG] sendAnswer called with: " + validationException);
    System.out.println("[DEBUG] phase=" + currentGamePhase + " ready=" + myUser.isReady() + " turnTaken=" + myUser.isTurnTaken());

    if (turnTaken) {
        try {
            ValidationUtil.requirePhase(currentGamePhase, phase.inProgress);
            ValidationUtil.requireUserIsNotReady(myUser);
            validationException = ValidationUtil.requireValidTurnOptions(validationException);
        }
        catch (ValidationException e) {
            LoggerUtil.INSTANCE.warning("Failed to send answer signal:" + e.getMessage(), Color.YELLOW);
            return;
        }

        Payload payload = new Payload();
        payload.setPayloadType(PayloadType.ANSWER);
        payload.setAnswer(answer);
        sendToServer(payload);
    }

    public void sendAnswerSignal(ValidationException validationException) {
        try {
            sendAnswer(answer);
        }
        catch (IOException e) {
            LoggerUtil.INSTANCE.warning("Failed to send answer signal:" + e.getMessage(), Color.YELLOW);
            return;
        }
    }
}
```

Shows send answer

 Saved: 5/7/2026 10:03:20 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for this game area's interactions
- Briefly explain the code flow for synchronizing authoritative state back to clients

Your Response:

GameView first receives the signal from when the user presses the button. This is then passed along to Client.java, which passes a method called "send_answer_signal", which is just identifying that the button was pressed since GameView and Client sort of exist in separate layers of function. This is then pushed into sendAnswer, which as was stated previously is then packaged into a payload of type Answer, which is then passed along to the server, which takes it in through processPayload, and then pushes it along to the needed method using CASE ANSWER..

handleAnswer() is responsible for validating the answer information, it does this by leveraging ValidationUtils and checking several logical passes, eg "is the user participating", "is the user not away", "is the phase in progress", and if it doesn't meet all of these checks, it will outright reject this information. Points, turns and phase are handled server-side, where the server decides what is true and simply sends down information



Saved: 5/7/2026 10:03:20 PM

Section #3: (4 pts.) Project Extra Features

Progress: 100%

≡ Task #1 (2 pts.) - Extra Feature 1

Progress: 100%

Details:

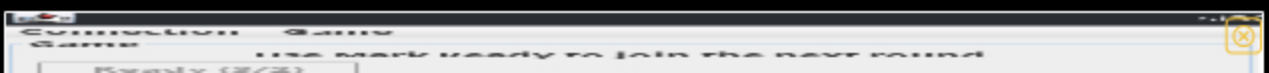
- Implement one project-specific Milestone 3 extra feature from your proposal
- Project hints:
 - Battleship: Currency System
 - Drawing: Drawing Options (colors + erase)
 - Go Fish: Extra Decks
 - Hangman: Restore Strikes option
 - RPS: Extra Choices (RPS-5)
 - Trivia: Category Selection
- If this feature is configurable, it should be session-creator controlled in Ready Check

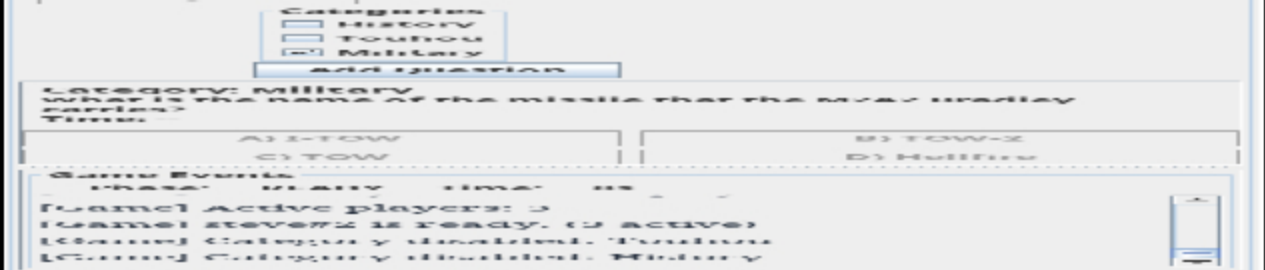
Part 1:

Progress: 100%

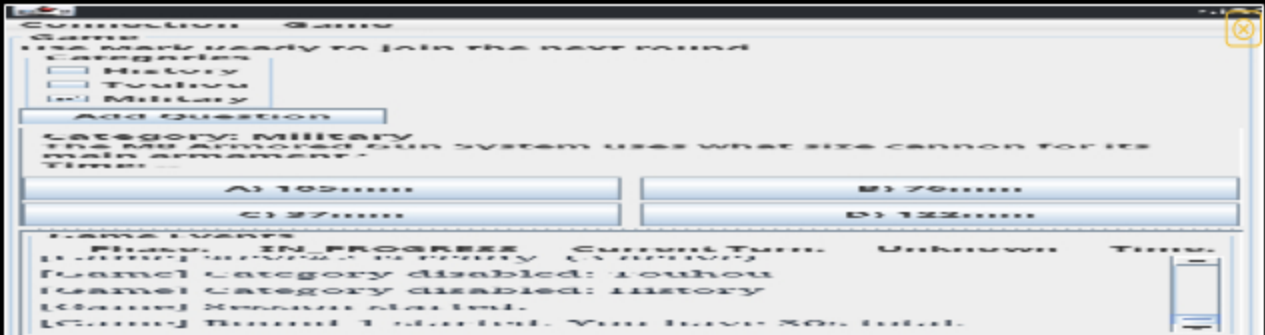
Details:

- Show the feature in action with 3+ clients visible when applicable
- Include code snippets for UI handling and server-side enforcement
- If this feature has host-only settings, show evidence of host-only access





This shows the various categories that can be selected



Shows that only military related questions are appearing



Only stefan#1 that started the session can use the panel

```
private void updateCategoryPanel() {
    Phase currentPhase = client.getCurrentGamePhase();
    boolean showCategories = currentPhase == Phase.READY && client.isSessionCreator();
    categoryPanel.setVisible(showCategories);

    if (!showCategories) {
        return;
    }

    categoryCheckboxes.removeAll();
    Set<String> available = client.getAvailableCategories();
    Set<String> enabled = client.getEnabledCategories();

    for (String category : available) {
        JCheckBox box = new JCheckBox(category, enabled.contains(category));
        box.addActionListener(e -> client.sendCategoryToggle(category));
        categoryCheckboxes.add(box);
    }

    categoryCheckboxes.revalidate();
    categoryCheckboxes.repaint();
    addQuestionButton.setVisible(currentPhase == Phase.READY);
}
```

updatecategorypanel method which checks to see if the client is the creator of the session, and then displays the categorypanel

```
private final JPanel categoryPanel = new JPanel();
private final JPanel categoryCheckboxes = new JPanel();
```

the initial invocation of the categorypanel variable which is later defined in GameView

```
public void handleCategoryToggle(ServerThread sender, String category) {
    try {
        ValidationUtils.requireTrue(sender.getClientId() == creatorClientId,
            errorMessage: "Only the session creator can change category settings.");
        ValidationUtils.requirePhaseAtMost(phase, Phase.READY);
        ValidationUtils.requireTrue(availableCategories.contains(category),
            "Unknown category: " + category);
        if (enabledCategories.contains(category)) {
            enabledCategories.remove(category);
            broadcastGameMessage("Category disabled: " + category);
        } else {
            enabledCategories.add(category);
            broadcastGameMessage("Category enabled: " + category);
        }
        broadcastCategories();
    } catch (ValidationException e) {
        LoggerUtil.INSTANCE.warning("[GameServer] " + e.getMessage());
        unicastGameMessage(sender, e.getMessage());
    }
}
```

GameServer.handleCategoryToggle; passes validations and then uses two lists previously created to decide what should be displayed

 Saved: 5/7/2026 10:18:17 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code for the feature and how it is enforced
- If applicable, briefly explain the code for host/session-creator configuration

Your Response:

When a user starts a session, they are given a specific variable that marks them as the "host" of that session. That ID is then checked against the various methods, eg on the client side GameView decides if it should display the categories depending on whether a boolean is filled, which is decided by client.isSessionCreator(). On the server side, there is a validation check right at the beginning, sender.getClientId is compared against the stored variable for the creatorClientId.

 Saved: 5/7/2026 10:18:17 PM

≡ Task #2 (2 pts.) - Extra Feature 2

Progress: 100%

Details:

- Implement a second project-specific Milestone 3 extra feature from your proposal
- Project hints:
 - Battleship: Proximity Sweep + Multishot actions
 - Drawing: Export Drawing (with delay window before clear)
 - Go Fish: Wildcards/Jokers
 - Hangman: Hard Mode
 - RPS: Choice cooldown

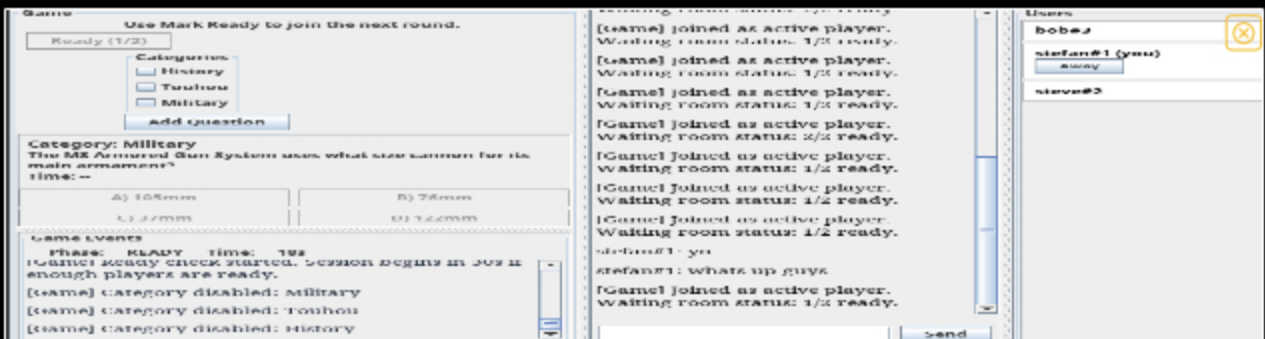
- Trivia: Add New Questions via UI and persist server-side
- If this feature introduces edge-case behavior, include evidence of those cases

Part 1:

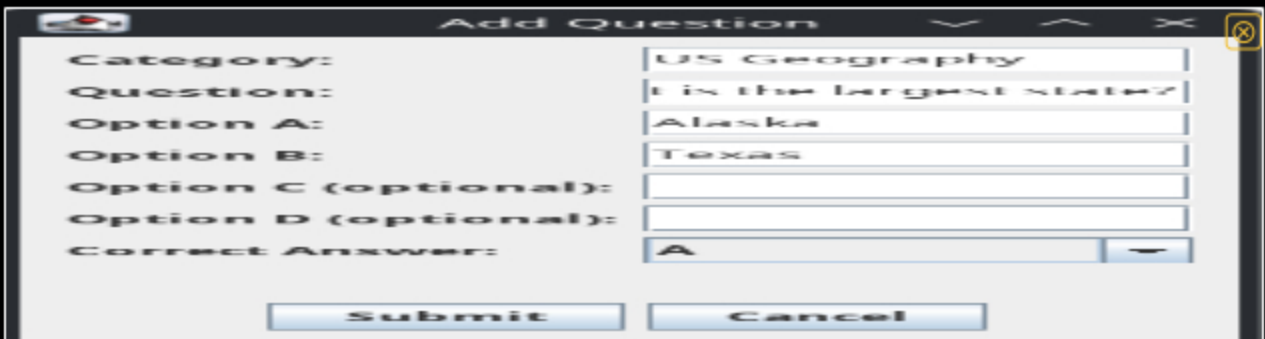
Progress: 100%

Details:

- Show a few examples of the feature in use
- Include code snippets showing core feature logic and validation
- Show at least one edge-case behavior (for example disabled option, invalid attempt blocked, or reset behavior)



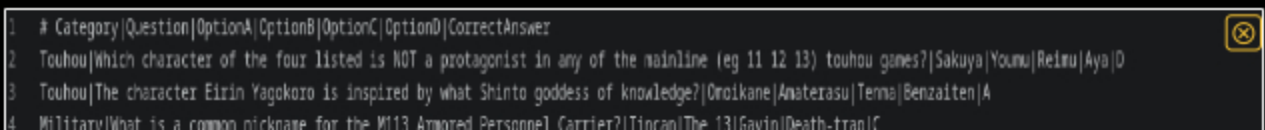
shows the host panel before using the feature



The question being created in the GUI



Question is added by the server



```

4 Military|What is the name of the missile that the M2A2 Bradley carries?|I-TOM|TOM-2|TOW|Hellfire|B
5 Military|The M8 Armored Gun System uses what size cannon for its main armament?|105mm|76mm|37mm|122mm|A
6 History|During the Korean war, what was the pivotal naval landing that led to the destruction of the PRK's army?|Incheon|Chipyong-ni|Osan|Chosin Reservoir|A
7
8
9
10 US Geography|What is the largest state?|Alaska|Texas|||A

```

lo and behold, question is added to questions.txt

Game

Use Mark Ready to join the next round.

Ready (1/2)

Categories

- ☒ US Geography
- ☐ History
- ☐ Touhou
- ☐ Military

Add Question

Question is added as category which means it's being indexed to be used

Category:

Question:

Option A:

Option B:

Option C (optional):

Option D (optional):

Correct Answer:

Category, question, and at least...

Submit Cancel

Error message from attempting to submit without inputting anything

```

public void sendAddQuestion(QuestionPayload payload) {
    try {
        sendToServer(payload);
    }
    catch (IOException e) {
        LoggerUtil.INSTANCE.warning("Failed to send add question: " + e.getMessage());
        emitUISystemMessage(message: "Failed to send question.");
    }
}

```

SendAddQuestion in Client.java,

```

public void handleAddQuestion(ServerThread sender, QuestionPayload payload) {
    try {
        ValidationUtils.requirePhaseAtMost(phase, Phase.READY);
        ValidationUtils.requireNotBlank(payload.getQuestionText(), errorMessage: "Question can't be blank.");
        ValidationUtils.requireNotBlank(payload.getCategory(), errorMessage: "Category can't be blank.");
        ValidationUtils.requireNotBlank(payload.getCorrectAnswer(), errorMessage: "Correct answer can't be blank.");
        ValidationUtils.requireNonNull(payload.getOptions(), errorMessage: "Options can't be null.");
        ValidationUtils.requireTrue(payload.getOptions().size() >= 2 && payload.getOptions().size() <= 4,
            errorMessage: "Must have 2 and 4 options");
        try (java.io.FileWriter fw = new java.io.FileWriter(QUESTIONS_FILE, true);
            java.io.BufferedReader bw = new java.io.BufferedReader(fw)) {
            StringBuilder line = new StringBuilder();

```

Handleaddquestion in Server.java

```
JButton submitButton = new JButton("Submit");
submitButton.addActionListener(e -> handleSubmit());
```

handlesubmit being invoked when the button is pressed

 Saved: 5/7/2026 10:42:02 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for this feature from UI to server-side and back to UI
- Briefly explain edge-case handling and reset behavior when applicable

Your Response:

In keeping with the rest of the design, I added a separate file for the separate window called AddQuestionDialog.java. When the submit button is pressed, the method called handleSubmit is called which is responsible for entering all the given information, performing validation (it will reject any bad input and return, after it checks to several conditions to ensure each of the fields are filled. This is then passed along to the client, which packages it up (it has its own payload type), which then passes it along to the server, which passes it through processpayload, and then invokes the method adds it to the list by writing into the file, etc.

Edge case behavior is handled as I showed in the screenshot, if for example the user doesn't enter a question, it will "fail" this condition if (category.isEmpty()), and return "statusLabel.setText("Category, question, and at least 2 options are required.");

 Saved: 5/7/2026 10:42:02 PM

Section #4: (2 pts.) Project General Requirements

Progress: 100%

≡ Task #1 (1 pt.) - Away Status

Progress: 100%

Details:

- Clients can mark themselves away and be skipped in active flow but still be part of the

Clients can mark themselves away and be skipped in active flow but still be part of the game/session

- The status should be visible to all participants
- A message should be relayed to the Game Events Panel
 - Example: "{displayName} is away" and "{displayName} is no longer away"
- The user list should have a visual representation (for example grayed out)
- Edge case: if the player still has pending actions and it is still their turn when away is triggered, this should end their turn

Part 1:

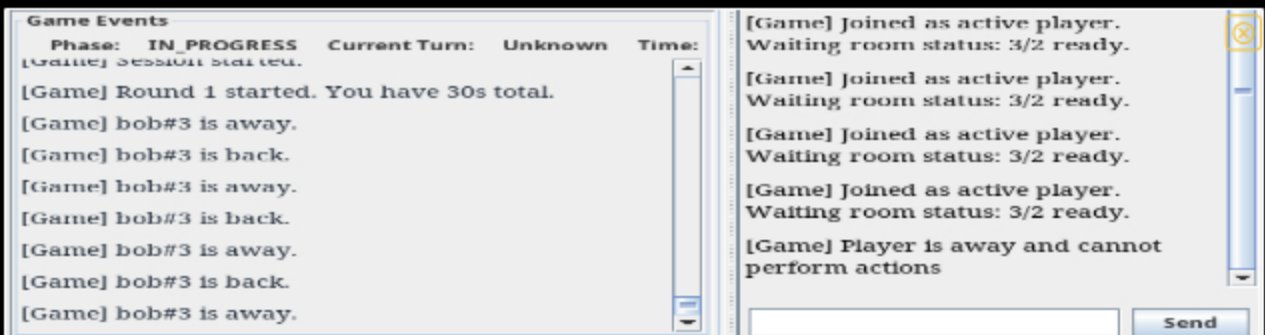
Progress: 100%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic
- Show one runtime example of the pending-action edge case ending the current turn



The away button



Shows you can't select anything if you're away

```
@Override
public void sendAwayToggle() throws ValidationException {
    try {
        Payload payload = new Payload();
        payload.setPayloadType(PayloadType.PLAYER_AWAY_STATUS);
        sendToServer(payload);
    } catch (ValidationException e) {
        // Handle exception
    }
}
```

```

    } catch (IOException e) {
        LoggerUtil.INSTANCE.warning("Failed to send away toggle: " + e.getMessage());
        emitUISystemMessage(message: "Failed to send away toggle.");
    }
}

```

Shows the send away toggle (signal is received from UI)

```

public void handleAwayToggle(ServerThread sender) {
    try {
        ValidationUtils.requireParticipating(isActivePlayer(sender),
            sender.getDisplayName() + " is not a participant");
        boolean away = !sender.isAway();
        sender.setAway(away);
        broadcastAwayStatus(sender.getClientId(), away);

        if (away) {
            broadcastGameMessage(sender.getDisplayName() + " is away.");
            // If the away player is currently taking a turn, advance to next player
            if (currentTurnPlayerId != null && currentTurnPlayerId == sender.getClientId()) {
                broadcastGameMessage(sender.getDisplayName() + " went away during their turn. Advancing turn.");
                onTurnEnd();
            }
        } else {
            broadcastGameMessage(sender.getDisplayName() + " is back.");
        }
    } catch (ValidationException e) {
        LoggerUtil.INSTANCE.warning("[GameServer] " + e.getMessage());
        emitGameMessage(sender, e.getMessage());
    }
}

```

sending the away signal

 Saved: 5/7/2026 10:55:58 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores away users from turn/round logic
- Briefly explain how the pending-action edge case is handled

Your Response:

Just as with the other methods, a signal is piped into Client.java, which prepares a payload, sends it to ServerThread, which processes it, invokes a method which exists in Server, and then that invokes a method that exists in GameServer that is responsible for handling the logic behind the away status. The sender of the payload is set as away in handleAwayToggle, and there is logic to simply skip over them if it comes to their turn.

 Saved: 5/7/2026 10:55:58 PM

≡ Task #2 (1 pt.) - Spectators

Progress: 100%

Details:

- Spectators are users who did not join as active participants for the current active session
 - Optionally you can include an explicit spectator toggle on Ready Check
- Spectators are ignored from active turn/round gameplay actions
- Spectators use spectator-only multicast chat to other spectators

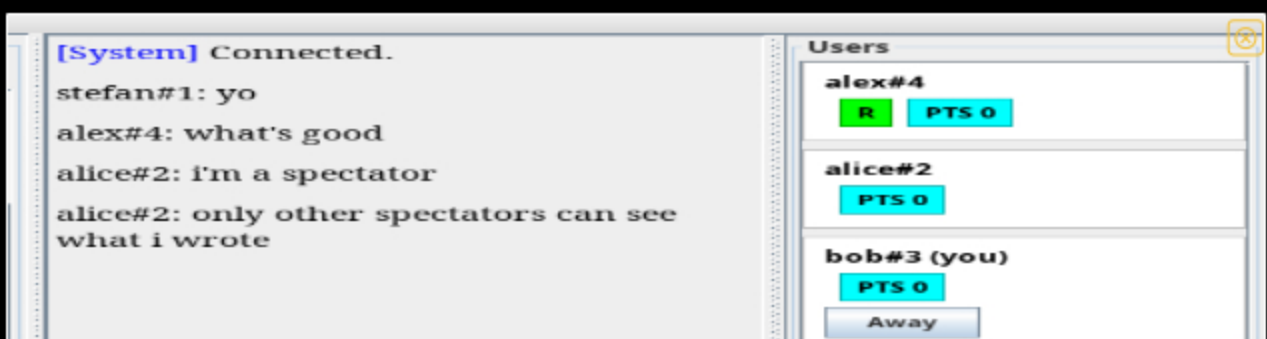
- Spectators cannot send active-player chat messages
- Spectators should have a visual representation in the user list when required by your project
- A message should be relayed to the Game Events Panel that a spectator joined during an in-progress session
 - Example: `"{displayName} joined as a spectator"`
- Spectators must wait until a new session to become active players
- On client joining during an active session, unicast Milestone 3 state not already covered in Milestone 2
 - Spectator status
 - Away status
 - Enabled session settings
 - UI should reflect proper state based on received data
- Project-specific spectator visibility hints:
 - Battleship: spectator can inspect player board state
 - Drawing: spectator can observe drawing state and current word
 - Go Fish: spectator can inspect player hand view but does not receive a hand
 - Hangman: spectator can see the correct word
 - Trivia: spectator can see the correct answer selected
 - RPS: spectator can observe session state and spectator chat restrictions

🖼️ Part 1:

Progress: 100%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending active-player chat messages (server-side)
- Show a mid-session join example where a spectator receives the necessary Milestone 3 join-state data
- Show the spectator's view of the session and project-specific visibility behavior



You can see they're spectators because they don't have the green R next to their name

[System] Connected.

[Game] joined as active player. Waiting room status: 1/2 ready.

stefan#1: yo

alex#4: what's good

Users

alex#4

R PTS 0

alice#2

PTS 0

bob#3

PTS 0

stefan#1 (you)

R PTS 0

Away

Players can't see what they're saying

```
// start region for lifecycle hook implementations
@Override
protected void onSpectatorJoined(ServerThread client) {
    if (client == null) {
        return;
    }
    if (phase == Phase.INACTIVE) {
        return; // nothing to sync if the session isn't active
    }
    LoggerUtil.INSTANCE.info("[GameServer] Spectator joined: " + client.getDisplayName());
    unicastGameMessage(client, "Joined as spectator. Current active players: " + getActivePlayerCount());
    unicastCategories(client);
    unicastCreator(client);
    unicastGameStateToJoiner(client);
}
```

on spectator join method

```
protected synchronized void broadcast(ServerThread sender, String message) {
    if (!isolateNonParticipantMessages
        && sender != null
        && isGameServerActive()
        && !gameServer.isActivePlayer(sender)) {
        // Non-participant that is multicast to other non-participants only.
        // Used to prevent non-participants from spoiling a game
        List<ServerThread> nonParticipants = getConnectedClientsSnapshot().stream()
            .filter(client -> client.getClientId() != sender.getClientId())
            .filter(client -> !gameServer.isActivePlayer(client))
            .collect(Collectors.toList());
        baseServer.multicastMessage(sender, message, nonParticipants);
        return;
    }
    baseServer.broadcast(sender, message);
}
```

server broadcast has an explicit check for this



Saved: 5/7/2026 11:06:14 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for spectator logic from server-side to UI
- Briefly explain how the server-side ignores spectators from turn/round logic
- Briefly explain spectator chat restrictions and enforcement
- Briefly explain how project-specific spectator visibility is implemented
- Briefly explain the mid-session join-state sync behavior

Your Response:

I have the baseline for this stuff but I unfortunately didn't have time to implement in the project

timeframe, since I'm writing this at 11:02 and I don't want to risk being behind on submission when I'm already technically past the deadline. However, this still semi-works, if you're a spectator and you send a message, only other spectators can see it because of how the broadcast is set up; it automatically filters your message to non-participants. This is still lacking the visual aspect and spectators can't see their own messages.

 Saved: 5/7/2026 11:06:14 PM

Section #5: (1 pt.) Misc

Progress: 100%

≡ Task #1 (0.33 pts.) - Github Details

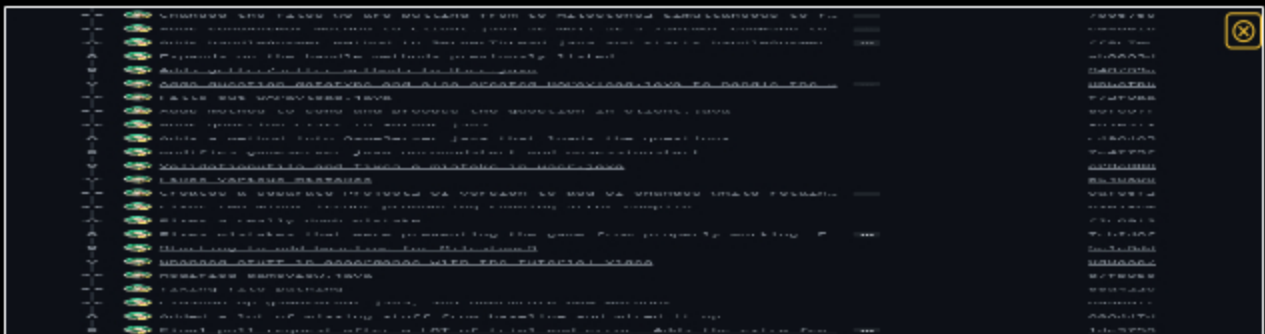
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



commit history (most of it)

 Saved: 5/7/2026 8:18:23 PM

↔ Part 2:

Progress: 100%

Details:

Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1

<https://github.com/methodius54/si329-it114-004-2026/pull/8>



URL

<https://github.com/methodius54/>

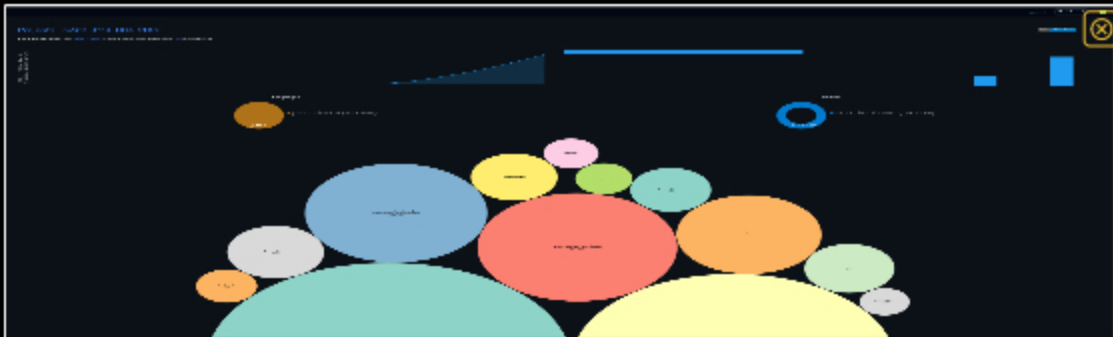
 Saved: 5/7/2026 8:18:23 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Shows the time and the repository name



Saved: 5/7/2026 8:19:43 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Throughout this project, I learned how networking works; how UIs work as well as how they all come together. There are several "layers", you might say between the inner "core" of methods and the buttons on top of the surface. I took Cybersec in school, and I remember the OSI model which sort of aligns with this perfectly, you have the top application layer with all the buttons and whistles, and then inside you have the presentation layer which transmits information in and out of the application layer. I realize that the explanation is not the best but this is what comes to mind from my personal experience and how it intertwines with this assignment.



Saved: 5/7/2026 8:15:56 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was probably just copying from the baseline in following with the tutorial. I am so deeply grateful for the tutorial by the way, because it made my life so much easier, since my structure somewhat aligned with click (you have a timeframe to do a task), it made it much easier to know what should be dropped (eg, the cards, grid, deck stuff) and what should be kept.



Saved: 5/7/2026 8:01:36 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was creating and coming up with the code for the extra features. While everything is still somewhat interoperable at this point, I ended up creating a separate class file in order to facilitate and handle the data required for changing the categories and questions.



Saved: 5/7/2026 8:02:52 PM